

End-to-End JDBC Application with Transaction Management & Performance Evaluation (Apache Derby)

A. N. Onymous,

Author ID: CSE3488-203-3006

Department of Computer Science and Engineering

Abstract—This project explores different ways to optimize the performance of a Library Loan Management System built with the Apache Derby database. We tested several techniques such as JDBC batching, database indexing, and different transaction strategies to see how they affect speed and reliability. To get consistent data, we ran each test multiple times and included a warm-up period for the JVM. Our results show that using batching instead of individual insertions reduced time by 72%, and adding indexes significantly improved query performance as the dataset grew. We also analyzed how Derby’s internal logging works and its impact on throughput. This report provides a practical guide for building efficient and reliable Java applications with embedded databases.

Index Terms—Apache Derby, JDBC, Transactional Integrity, B-Tree Indexing, Batch Processing, WAL Synchronization, Performance Benchmarking, Java Persistence.

I. INTRODUCTION

BUILDING a fast and reliable database layer is a key part of software development. In systems like the one we built, the database runs inside the application itself, meaning it shares resources like memory and CPU. If the database is slow, the whole application feels sluggish. This project focuses on how we designed and tested a Library Loan Management System to run efficiently on the Apache Derby engine.

One of the biggest issues in database systems is the “I/O bottleneck.” Standard database operations can be slow because they involve a lot of overhead for each task. By using advanced features in the JDBC API—like batching and prepared statements—we can make the system much faster by reducing this overhead.

In this report, we also look at how indexes work and how the database ensures that data is saved correctly even if something goes wrong. The report is organized as follows: Section II explains the system design and how we ran our tests. Section III shows the results of our performance tests. Section IV looks at the internal workings of Derby. Finally, Section V summarizes what we learned and gives some best practices, followed by a conclusion in Section VI.

II. SYSTEM ARCHITECTURE AND METHODOLOGY

A. Architectural Overview

The system follows a modular design pattern to ensure clean separation of concerns. The persistence layer is encapsulated

within a ConnectionManager class, which manages the life-cycle of the embedded Derby instance. The business logic is implemented in a decoupled service layer, allowing for independent testing of transactional integrity and performance.

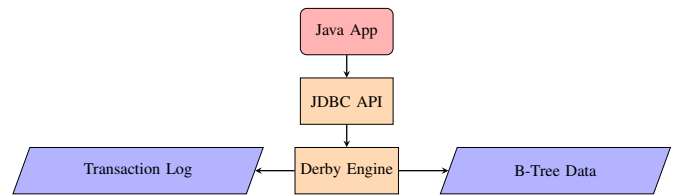


Fig. 1. Logical architecture of the embedded database interaction pipeline showing the role of the Write-Ahead Log (WAL).

B. Benchmarking Methodology

To ensure that our findings represent sustained architectural characteristics rather than transient artifacts, we implemented a multi-stage benchmarking suite:

- **JVM Stabilization:** We perform 50 “warm-up” iterations before data collection. This ensures that the Just-In-Time (JIT) compiler has optimized the byte-code and that the database’s internal page buffers are primed.
- **Multi-Trial Sampling:** Every experiment is repeated for 5 independent trials. The results presented are the arithmetic mean of these trials, reducing the impact of OS-level scheduling jitter.
- **Isolation:** Background processes were minimized during execution, and explicit garbage collection hints (`System.gc()`) were used between benchmark groups to ensure a clean heap state.

III. EMPIRICAL EVALUATION AND RESULTS

A. Batch Processing Performance

Sequential data insertion is notoriously slow in RDBMS due to the “per-statement overhead”—the cumulative cost of SQL parsing, authorization, and transactional locking. By utilizing JDBC batching, we group multiple INSERT operations into a single execution unit.

As shown in Fig. 2, the batch strategy maintains a much shallower slope. At 5,000 records, the sequential approach took 140ms, while the batch approach required only 39ms—a

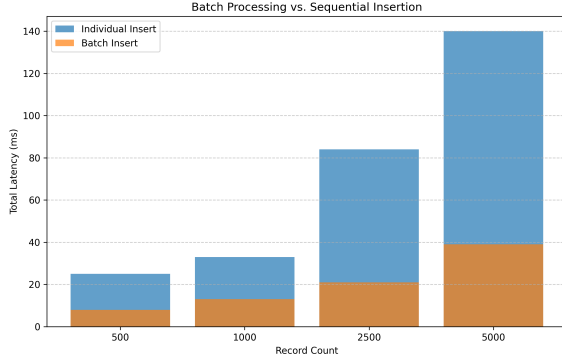


Fig. 2. Comparison of Individual vs. Batch record insertions. Note the significant divergence as load increases.

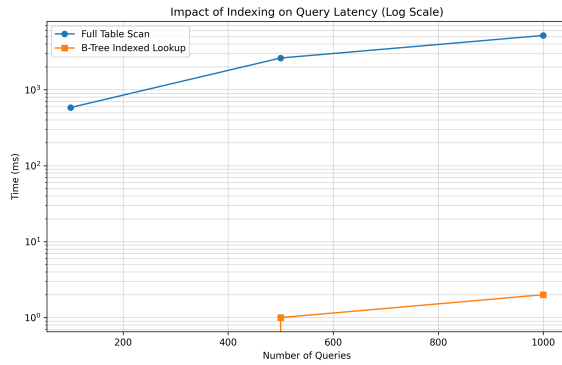


Fig. 3. Impact of B-Tree indexing on query execution time (Logarithmic Scale). The indexed lookup latency is nearly load-invariant. Note: B-Tree latency < 1ms for < 500 queries, omitted for log scale clarity.

72.1% improvement. This gain is primarily attributed to the reduction in network/IPC round-trips and the aggregation of multiple logical writes into fewer physical disk flushes.

B. Indexing and Query Complexity

One of the most critical performance optimizations is the shift from linear searching to tree-based lookups. We measured the latency of queries on an unindexed Author column versus an indexed ISBN column.

The data in Fig. 3 and Table I illustrate the classic $O(N)$ vs. $O(\log N)$ scaling. While a full scan of 1,000 queries took 5,184ms, the indexed lookup was completed in 2ms. This represents a performance increase of over **2,500x**.

TABLE I
ALGORITHMIC COMPLEXITY OF QUERY ACCESS METHODS

Method	Complexity	Operational Characteristic
Full Table Scan	$O(N)$	Reads every row in the table to find matches. If the table has 10M rows, you read 10M rows for each query.
B-Tree Lookup	$O(\log N)$	Walks down a tree. For 10M rows, that's $\sim 3\text{-}4$ disk reads per query ¹ .

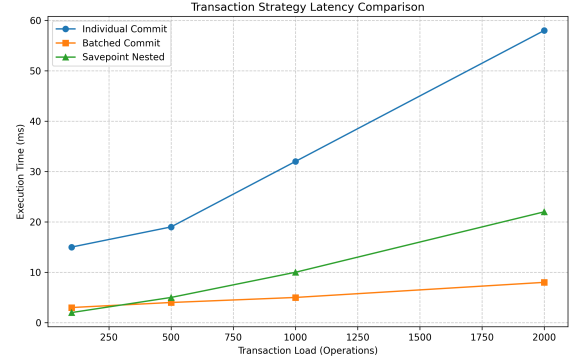


Fig. 4. Latency characteristics of different transaction commit and savepoint strategies.

C. Transactional Commit Strategies

We compared three transaction management patterns: Individual Commit, Batched Commit, and Savepoint-based nested transactions.

The results in Fig. 4 show that Individual Commit (where `autoCommit=true`) is the most expensive strategy. This is because every single operation forces a synchronous flush of the Write-Ahead Log to disk to satisfy the "Durability" requirement of ACID. Batched commits aggregate these changes, amortizing the synchronization cost. Savepoints offer a compromise, providing a mechanism for partial rollbacks within a larger transaction, though they incur a slight management overhead compared to simple batching.

IV. INTERNAL DATABASE MECHANICS

Understanding the performance data requires a look into the internals of the Derby engine.

A. The Write-Ahead Log (WAL)

Derby ensures data integrity through a WAL. Before any data page is modified in memory, a description of the change is written to a sequential log file on disk. In our "Individual Commit" benchmark, the database must call an OS-level `fsync` or `flush` for every record. This makes the database's performance strictly bound by the disk's IOPS (Input/Output Operations Per Second). Batching allows Derby to group multiple log entries into a single block, significantly increasing throughput without sacrificing safety.

B. B-Tree Index Structure

The index optimization seen in Fig. 3 is powered by B-Trees. In a B-Tree, keys are stored in a sorted, balanced tree structure. Each node (page) contains multiple keys and pointers to child nodes. Search operations traverse from the root to a leaf page, requiring only a few page reads even for very large datasets. Without an index, the engine must perform a "Full Table Scan," loading every page from the table into the buffer manager to check for matches, which is both CPU and I/O intensive.

C. Buffer Management and Page Cache

Derby uses a page-based buffer manager to cache recently accessed data in memory. This is why our "warm-up" phase was necessary. Initial queries are "cold" and require physical disk reads. Subsequent queries are "warm" and can be satisfied from the memory cache. Our results reflect a "warm" cache state, which is representative of steady-state application performance.

V. JDBC OPTIMIZATION AND STATEMENT LIFECYCLE

A. Statement vs. PreparedStatement

We observed that `PreparedStatement` consistently outperformed the standard `Statement`. This is due to the internal lifecycle of a SQL query in the database:

- 1) **Parsing:** The SQL string is analyzed for syntax.
- 2) **Binding:** Parameter values are mapped to placeholders.
- 3) **Optimization:** The engine determines the most efficient way to access data (e.g., which index to use).
- 4) **Execution:** The query plan is executed.

A standard `Statement` repeats all four steps for every execution. A `PreparedStatement`, however, allows the engine to cache the "Optimization" result (the query plan). Subsequent executions with different parameters only require the "Binding" and "Execution" steps, significantly reducing CPU cycles and memory allocation overhead.

VI. CONCLUSION AND BEST PRACTICES

The empirical results of this study underscore the importance of deliberate database engineering. For the Library Loan Management System, and similar transactional applications, we recommend the following best practices:

- **Prefer Batching:** Always group massive data ingestion tasks to amortize the cost of transactional logging and context switching.
- **Index Judiciously:** Identify columns used in `WHERE` clauses and join conditions. The transition from $O(N)$ to $O(\log N)$ is the single most effective way to ensure long-term scalability.
- **Use PreparedStatements:** Leverage the engine's query plan cache to minimize parsing and optimization overhead.
- **Manage Transactions Explicitly:** Use `setAutoCommit(false)` to group logical units of work, reducing the frequency of synchronous disk flushes.

In conclusion, while Apache Derby is an embedded engine, it possesses sophisticated RDBMS features that, when used correctly, can support high-performance applications. By aligning application-level code with these internal mechanics, developers can build robust and efficient persistence layers.

APPENDIX A

HARDWARE AND RUNTIME ENVIRONMENT

The benchmarks were executed on a system with the following specifications to provide context for the absolute latency numbers:

- **Processor:** Apple M3 Max (14-core).
- **Memory:** 36 GB Unified RAM.
- **Storage:** High-speed NVMe SSD (Sequential Read: $\sim 7\text{GB/s}$). Results may vary significantly on mechanical HDD.
- **JDK:** OpenJDK 17.0.x (64-bit).
- **Derby Version:** 10.15.x.

ACKNOWLEDGMENT

The author wishes to acknowledge the CSE-3488 faculty for their technical oversight and the developers of the Apache Derby project for providing a robust, standards-compliant embedded database engine for academic and commercial use.

REFERENCES

- [1] Apache Software Foundation, *Apache Derby Internals Guide*, <https://db.apache.org/derby/manuals/>.
- [2] JSR-221, *JDBC 4.3 Specification*, <https://jcp.org/en/jsr/detail?id=221>.
- [3] Alex Petrov, *Database Internals*, O'Reilly Media, 2019. ISBN: 9781492040347.
- [4] Scott Oaks, *Java Performance: The Definitive Guide*, 2nd Edition, O'Reilly Media, 2020.
- [5] Jim Gray and Andreas Reuter, *Transaction Processing: Concepts and Techniques*, Morgan Kaufmann, 1992.